



Софийски университет „Св. Климент Охридски“

Факултет по математика и информатика

КУРСОВ ПРОЕКТ

по Обектно-ориентирано програмиране

Приложение за работа с електронни таблици

Изготвил: **Антон Вачков**

Специалност: **Компютърни науки**

Съдържание

1	Увод	2
1.1	Описание и идея на проекта	2
1.2	Цел и задачи на разработката	2
1.3	Структура на документацията	3
2	Преглед на предметната област	4
2.1	Основни дефиниции, концепции и алгоритми	4
2.2	Проблеми и сложност	4
2.3	Методи за решаване на проблемите	4
2.4	Потребителски и качествени изисквания	5
3	Проектиране	6
3.1	Обща архитектура – ООП дизайн	6
3.2	Диаграми и извадки от кода	7
4	Реализация и тестване	9
4.1	Реализация на класове	9
4.2	Управление на паметта, алгоритми и оптимизации	10
4.3	Планиране и тестови сценарии	11
5	Заключение	13
5.1	Обобщение на изпълнението на началните цели	13
5.2	Насоки за бъдещо развитие и усъвършенстване	13
	Използвана литература	14

Глава 1

Увод

1.1 Описание и идея на проекта

Основната идея на проекта е създаването на конзолно приложение на езика C++ за управление на електронни таблици. Програмата предоставя текстов интерфейс, чрез който потребителят може да зарежда данни от файл в паметта, да редактира съдържанието на отделните клетки и да визуализира таблицата. Поддържа се работа с различни типове данни в една и съща таблица, както и динамично изчисляване на математически формули с препратки към други клетки. Всички промени се запазват обратно в текстов файл при изрично желание на потребителя.

Проектът е с отворен код и достъпен в GitHub на адрес:
<https://github.com/AVachkov/Spreadsheet>.

1.2 Цел и задачи на разработката

Целта на проекта е да се изгради работеща система, която абстрахира сложността на вътрешното представяне на данните и се фокусира изцяло върху същността на тяхната обработка, изчисляване и сигурно съхранение.

Основните задачи, решени по време на разработката, са:

- **Структура на данните:** Създаване на общ базов клас и отделни класове за поддържаните хетерогенни типове съдържание: цели числа, дробни числа, символни низове и формули.
- **Взаимодействие между класовете:** Правилно организиране на връзките и комуникацията между главния клас на таблицата и отделните обекти за клетките, за да работят заедно като единна система.

- **Пресмятане на формули:** Реализиране на алгоритъм, който чете математически изрази, поддържа основните аритметични операции (+, -, *, /, ^), спазва техния приоритет и преобразува типовете при нужда (например празни клетки и нечислови низове се оценяват като 0).
- **Изчисляване на препратките:** Извличане на стойности от конкретни адреси (например R1C1), като при грешка във формулата или деление на нула програмата не прекъсва своето изпълнение, а просто извежда "ERROR" в съответната клетка.
- **Защита от зацикляне:** Справяне с проблема, при който две клетки зависят една от друга, за да се предотврати блокиране на програмата.
- **Управление на паметта:** Внимателно и оптимално заделяне и освобождаване на динамична памет, за да няма утечки (*memory leak*).
- **Работа с файлове:** Реализиране на команди за зареждане (*open*) и записване (*save*, *saveas*) на таблицата.

1.3 Структура на документацията

Настоящата документация е организирана в пет основни глави:

- **Глава 1 - Увод:** Описва идеята на проекта, целите и основните решени задачи.
- **Глава 2 - Преглед на предметната област:** Разглежда дефинициите за типовете данни, формулите и функционалните изисквания.
- **Глава 3 - Проектиране:** Представя архитектурата на приложението и ООП дизайна.
- **Глава 4 - Реализация и тестване:** Описва детайли по управлението на паметта, алгоритмите и проведените тестове.
- **Глава 5 - Заключение:** Обобщава постигнатите резултати и дава насоки за бъдещо развитие.

Глава 2

Преглед на предметната област

2.1 Основни дефиниции, концепции и алгоритми

Електронната таблица се моделира като правоъгълна матрица в оперативната памет. Основните градивни единици са клетките, които могат да бъдат празни или да съдържат: числа, текст (в кавички) или формули (започващи с =). За пресмятане на формулите и извличане на стойности от други клетки се използва рекурсивен алгоритъм.

2.2 Проблеми и сложност

Основно предизвикателство е как формулите с препратки да извличат нужните стойности, без да се създава силна зависимост към цялостната структура на таблицата.

Друг съществен проблем възниква при зареждането на данните. Динамичното изчисляване на формули не може да се извършва линейно по време на четенето от файл, тъй като те могат да реферират към клетки, които все още не са заредени в паметта.

2.3 Методи за решаване на проблемите

За справяне с дефинираните алгоритмични предизвикателства са приложени следните подходи:

- **Взаимодействие и достъп до данни:** За да се избегне подаването на директен достъп до целия масив с данни, се използва механизъм за индиректно извличане на стойности. По този начин формулите получават нужните данни в реално време, без да се нарушава капсулацията на таблицата.

- **Отложено изчисляване на формули:** Първо цялата таблица се зарежда напълно в паметта, а след това за извличане на липсващи стойности от реферирани клетки се използва рекурсия. За да се предотврати безкраен цикъл при взаимнозависими клетки (циклични препратки), преди всяко извикване временно се задава статус на активно изчисляване. При повторно засичане на такъв статус, алгоритъмът разпознава зациклянето и безопасно прекъсва операцията.

2.4 Потребителски и качествени изисквания

Функционални изисквания (Потребителски): Приложението предоставя конзолен интерфейс (CLI). Таблиците се зареждат от текстов файл (стойности, разделени със запетаи). Потребителят разполага със следните команди:

- Файлови: `open <file>`, `close`, `save`, `saveas <file>`, `help`, `exit`.
- Операционни: `edit <address> <value>` (промяна на клетка) и `print` (отпечатване с автоматично подравняване).

Качествени изисквания (Нефункционални): Системата трябва да осигурява висока стабилност при невалидни данни – да конвертира празни и текстови клетки до 0 при изчисление, и да индикира грешки (напр. деление на нула) без срив на програмата. Приложението трябва да е устойчиво на неточно въвеждане, като автоматично игнорира излишни интервали около разделителите.

Глава 3

Проектиране

3.1 Обща архитектура – ООП дизайн

Архитектурата на приложението е изградена на базата на обектно-ориентираното програмиране, като логиката е разпределена между главния клас `Spreadsheet` и йерархията на класа `Cell`.

Клас `Spreadsheet`: Това е главният клас, който капсулира двумерната матрица от указатели към базовия клас `Cell`. Той не само предоставя публичен интерфейс за основните команди (зареждане, запис, редактиране, отпечатване), но и изцяло управлява жизнения цикъл на клетките (динамичната памет). В неговите отговорности влизат още десериализацията на входните данни и стартирането на процеса по разрешаване на зависимостите във формулите.

Йерархия на клетките (`Cell`): Всяка клетка е представена чрез базов клас `Cell`, който задава общи задължителни виртуални методи (за отпечатване, извличане на дължина на съдържанието и извличане на числова стойност). За различните видове данни са създадени конкретни класове-наследници: за цели числа (`WholeNumberCell`), дробни числа (`DecimalNumberCell`), текст (`TextCell`), формули (`FormulaCell`), празни клетки (`EmptyCell`) и грешки (`ErrorCell`).

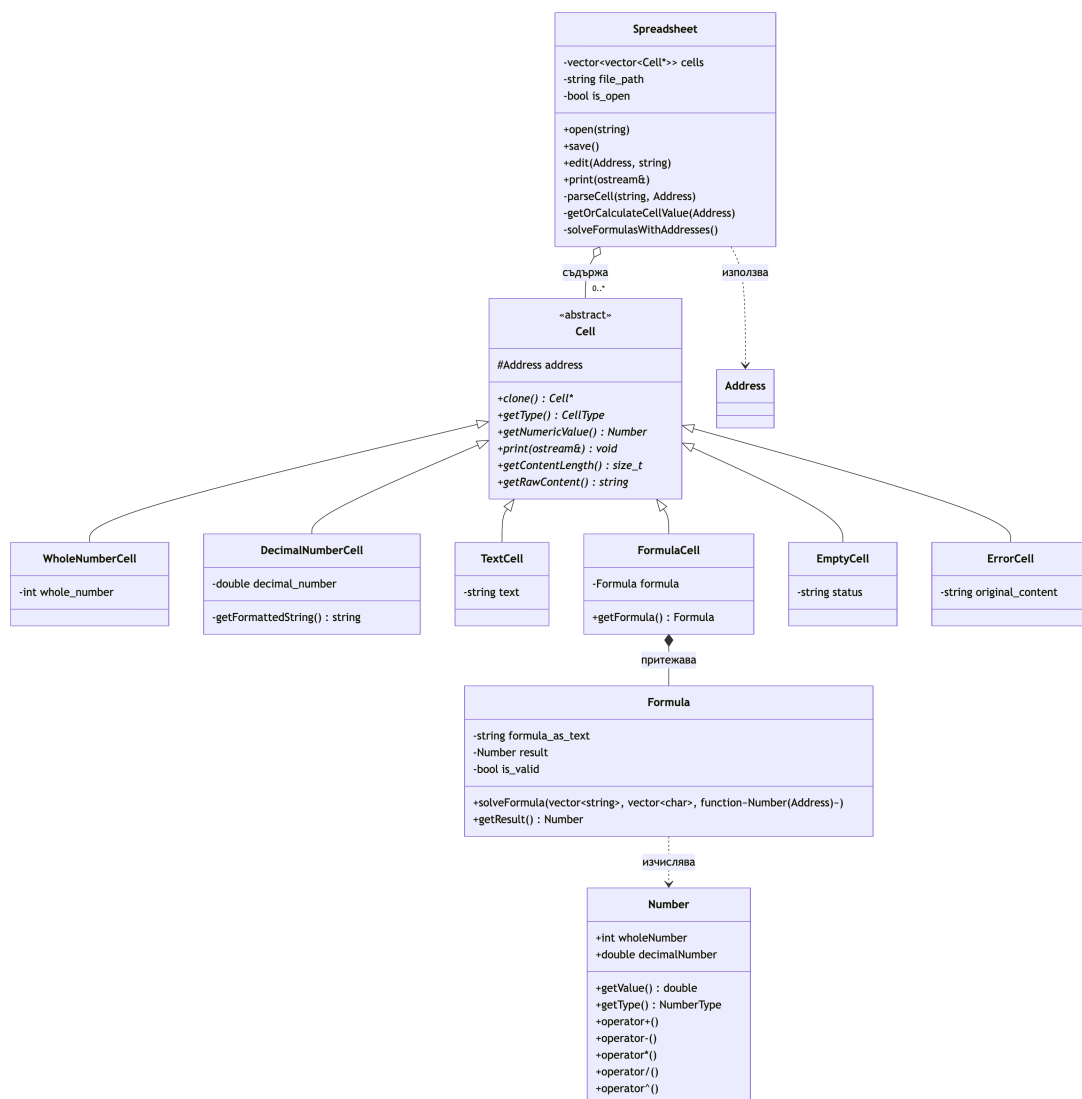
Изчисляване на формули (`Formula` и `FormulaCell`): Класът `FormulaCell` съдържа в себе си обект от тип `Formula`, който изцяло поема логиката по анализиране и пресмятане на математически изрази. При проектирането на тази връзка е избегната първоначалната идея за подаване на директен указател от формулата към таблицата, тъй като това би обвързало класовете твърде силно. Вместо това е внедрено по-елегантно решение с `std::function`. Методът `solveFormula` извършва пресмятанията над отделните компоненти на израза, като разчита на подадена от `Spreadsheet` ламбда функция за индиректно извличане на стойности в реално време. По този начин `Formula` остава напълно

капсулиран, изолиран от вътрешната структура на матрицата и независим от нейната организация.

3.2 Диаграми и извадки от кода

Диаграма на класовете

Структурата на класовете и полиморфните връзки между главния мениджър **Spreadsheet**, базовия абстрактен клас **Cell** и неговите конкретни наследници са представени на фиг. 3.1.



Фигура 3.1: Структурна диаграма на класовете

Поведенчески модел: Индиректно извличане на стойности

Следната извадка от метода `getOrCalculateCellValue` демонстрира ключовия поведенчески модел – как `Spreadsheet` обработва формулите и им подава ламбда функция, улавяйки указателя `this`. Това позволява на обекта `Formula` да извлича стойности от други клетки в реално време. Преди извикването, клетката временно получава статус "CALCULATING", за да се предотврати безкрайна рекурсия:

```
1 std::vector<std::string> tokens;
2 std::vector<char> operators;
3
4 std::string formulaText = formula.getFormulaText();
5 tokenize(formulaText, tokens, operators);
6
7 // Временно заместване за защита от циклични препратки
8 delete cells[a.row - 1][a.col - 1];
9 cells[a.row - 1][a.col - 1] = new EmptyCell(a, "CALCULATING");
10
11 try
12 {
13     // Подаване на ламбда функция, чрез която формулата
14     // получава възможност индиректно и рекурсивно да
15     // извлича непресметнати стойности
16     formula.solveFormula(tokens, operators,
17         [this](Address a) {
18             return this->getOrCalculateCellValue(a);
19         });
20
21 }
22 catch (const std::runtime_error &e)
23 {
24     delete cells[a.row - 1][a.col - 1];
25     cells[a.row - 1][a.col - 1] = new ErrorCell(a, formulaText);
26     throw;
27 }
```

Глава 4

Реализация и тестване

4.1 Реализация на класове

В основата на реализацията на електронната таблица стои полиморфният контейнер (правоъгълна матрица от клетки), управляван от класа `Spreadsheet`. Тъй като таблицата трябва да съхранява хетерогенни данни (текст, числа и формули) в обща структура, тя е реализирана като двумерен вектор от указатели към базовия абстрактен клас `Cell`:

```
1 // Декларация на основния контейнер в класа Spreadsheet
2 std::vector<std::vector<Cell *>> cells;
```

Използването на указатели към базовия клас позволява прилагането на полиморфизъм. Когато `Spreadsheet` изпълнява команди като отпечатване или изчисляване, той извиква виртуалните методи (като `print()` или `getNumericValue()`) директно през общия интерфейс на `Cell`. По този начин главният клас управлява всички видове клетки по абсолютно еднакъв начин, без нуждата да проверява ръчно дали конкретният обект зад указателя е число, текст или формула.

За да се попълни тази матрица с реални обекти при десериализация или редактиране, е имплементиран частният метод `parseCell`. Тъй като останалата част от логиката работи само с абстрактни указатели, този метод служи като мост – той директно приема текстовия низ, разпознава типа му чрез помощни функции и заделя памет за съответния наследник:

```
1 // Метод за разпознаване и създаване
2 // на конкретни обекти клетки
3 Cell* parseCell(const std::string &token, Address a);
```

По този начин процесът по разпознаване на типовете е капсулиран на едно единствено място в кода, което улеснява бъдещото добавяне на нови видове клетки.

Реализация на връзката между таблицата и формулите

Едно от основните имплементационни предизвикателства е взаимодействието между класа `Spreadsheet` и обектите от тип `Formula`. Първоначалната идея за осигуряване на достъп на формулите до клетките беше подаване на директен указател към таблицата, за да се предотврати копирането на целия двумерен вектор. Тъй като обаче формулите нямат реална нужда от достъп до всички клетки, бе реализирано по-елегантно решение чрез `std::function`.

Основният механизъм, който прави това възможно, е т.нар. *capture list* (списък за улавяне) в C++. Чрез него лямбда изразът в `solveFormula` „улавя“ указателя към текущата инстанция на таблицата (`[this]`) и го предава индиректно:

```
1 // Ламбда функция за индиректно извличане на стойности
2 formula.solveFormula(tokens, operators, [this](Address a) {
3     return this->getOrCalculateCellValue(a);
4 });
```

По този начин обектът `Formula` извлича стойности от адреси в реално време, без изобщо да се интересува от структурата на двумерния масив и без да нарушава капсулацията на `Spreadsheet`.

Използван външен код

Логиката във функцията `TextCell::getContentLength()` във файла `cell.cpp` е взета от генеративен модел, тъй като стандартният метод `std::string::length()` не работи коректно за текст на кирилица.

4.2 Управление на паметта, алгоритми и оптимизации

Управление на ресурсите чрез „Голямата четворка“

Тъй като `Spreadsheet` управлява вектор от указатели към динамично заделена памет, в него е реализирана „Голямата четворка“ (*Big Four*) – дефинирани са конструктор по подразбиране, копиращ конструктор, оператор за присвояване и деструктор. За нуждите на копирането се използва виртуалният метод `clone()` от `Cell`. Всеки наследник връща свое дълбоко копие (*deep copy*), което елиминира риска от споделена памет и предпазва от „двойно освобождаване“ (*double-free*).

Полиморфно разрушаване на обекти

Задължителен елемент от йерархията е дефинирането на виртуален деструктор в базовия

клас `Cell`. Той гарантира, че при разрушаване на клетка през указател от базов тип, правилно ще се извърши полиморфно извикване на деструктора на конкретния наследник, предотвратявайки изтичане на памет (*memory leak*).

Изчисляване на формули и защита от безкрайна рекурсия

Изчисляването на формули с препратки към други клетки изисква таблицата да бъде предварително заредена в паметта, тъй като даден израз може да реферира към клетка, която все още не е пресметната. Поради този факт стандартното линейно изчисляване по време на самото четене е неприложимо. Процесът е разделен на два етапа: първо се зареждат всички независими клетки (числа, текст и формули без препратки), след което се прилага стратегия за отложено изчисляване – при срещане на препратка във формула, алгоритъмът се извиква рекурсивно за съответната клетка, за да извлече стойността ѝ в реално време.

За да се предотврати безкрайна рекурсия при циклични зависимости (напр. две клетки, които взаимно се реферират), преди всяко рекурсивно извикване текущата клетка се замества с `EmptyCell` и се маркира временно със статус "CALCULATING" (това е конкретното предназначение на член-данната `status` в `EmptyCell`). При повторен опит за достъп до клетка в това състояние се индикира зацикляне и изпълнението се прекъсва безопасно чрез хвърляне на изключение.

Оптимизация на матрицата

При четене от файл редовете често съдържат различен брой клетки, което прави структурата в паметта „назъбена“ (*jagged*). Методът `alignAllCells` допълва матрицата с празни клетки (`EmptyCell`) до изравняване на всички редове спрямо най-дългия. Това осигурява правилна правоъгълна структура и елиминира риска от грешки при индексирание извън границите.

4.3 Планиране и тестови сценарии

Коректността на системата е проверена чрез примерната таблица, показана на табл. 4.1.

10	20	30	40
10		1000	
	10		

Таблица 4.1: Начално състояние на тестовата таблица

Върху нея през конзолния интерфейс са изпълнени сценарии, покриващи нормалната работа и защитите от грешки:

- **Сценарий 1: Изчисляване на формула с валидни адреси**

Вход: edit R2C1 =R1C1+R1C2

Резултат: Системата успешно събира стойностите (10 и 20). При отпечатване клетката извежда 30.

- **Сценарий 2: Операция с празна клетка**

Вход: edit R2C2 =R3C1+R3C2

Резултат: Тъй като R3C2 е празна (интерпретира се като 0), резултатът от израза се оценява коректно на 10.

- **Сценарий 3: Защита от деление на нула**

Вход: edit R4C1 =R1C4/R2C4

Резултат: Клетката R2C4 е празна (0). Математическата валидация улавя делението на нула, предпазва програмата от срыв и клетката връща ERROR.

- **Сценарий 4: Защита от циклична зависимост**

Вход: edit R2C1 =R3C3+R2C1

Резултат: Алгоритъмът засича статус "CALCULATING" заради самореференцията, прекъсва цикъла безопасно.

Глава 5

Заклучение

5.1 Обобщение на изпълнението на началните цели

Проектът успешно изпълни началните си цели, реализирайки работеща конзолна електронна таблица, която обработва различни типове данни и формули.

5.2 Насоки за бъдещо развитие и усъвършенстване

Основна насока е добавянето на номера на редове и колони при принтиране за по-добра прегледност в конзолата. Следваща ключова стъпка е разработването на графичен потребителски интерфейс (GUI), който да замени изцяло текстовия команден ред.

Използвана литература

- [1] Георгиев, Калин. *Задачи за самоподготовка по Увод в програмирането, Обектно-ориентирано програмиране, Структури от данни и Функционално програмиране*. Факултет по математика и информатика, Софийски университет „Св. Климент Охридски“, 2026.
- [2] Георгиев, Калин. *Лекции по Обектно-ориентирано програмиране (lecture-notes)*. GitHub, https://github.com/stranxter/lecture-notes/tree/master/notes/02_oop.
- [3] Лекова, Мая. *Материали по Обектно-ориентирано програмиране (oop-fmi)*. GitHub, <https://github.com/MayaLekova/oop-fmi>.
- [4] Костадинов, Теодор и Александър Мандев. *Материали по Обектно-ориентирано програмиране (Object_oriented_programming_FMI)*. GitHub, https://github.com/tkostadinov004/Object_oriented_programming_FMI.
- [5] “C++ Reference.” *cppreference.com*, <https://en.cppreference.com/>.
- [6] “C++ Standard Library.” *cplusplus.com*, <https://cplusplus.com/reference/>.